Chapter6 Normalization and Functional Dependencies

1 A Single Table Approach

Imagine a university keeps all student enrollment data in a **single table** named `Enrollment`:

StudentID	StudentName	CourseID	CourseTitle	Instructor	Grade
101	Ahmed	CS101	Intro to CS	Dr. Sameer	A
101	Ahmed	MATH201	Calculus II	Dr. Ibrahem	B+
102	Sami	CS101	Intro to CS	Dr. Layla	A-
102	Sami	PHY301	Quantum Physics	Dr. Amani	C

> ⚠ **The Problem**
>
> This looks simple — but it contains serious problems known as **anomalies**.

2 Anomalies in Unnormalized Tables

2.1 Insertion Anomaly

Problem: Cannot insert certain data without other unrelated data.

Suppose a new course `CS202 – Data Structures` is added, but no student has enrolled yet. You cannot insert `CourseID = CS202` and `CourseTitle = Data Structures` unless you also have a `StudentID`. The course information depends on a student being enrolled.

StudentID	StudentName	CourseID	CourseTitle	Instructor	Grade
101	Ahmed	CS101	Intro to CS	Dr. Layla	A
101	Ahmed	MATH201	Calculus II	Dr. Ibrahem	B+
102	Sami	CS101	Intro to CS	Dr. Layla	A-
102	Sami	PHY301	Quantum Physics	Dr. Amani	C
Null	**Null**	**CS202**	**Data Structures**	**Null**	**Null**

2.2 Deletion Anomaly

Problem: Deleting data unintentionally removes other important data.

If Ahmed drops `MATH201`, and he was the only student in that course, deleting his record also deletes the course info (`MATH201`, `Calculus II`, `Dr. Ibrahem`).

StudentID	StudentName	CourseID	CourseTitle	Instructor	Grade
101	Ahmed	CS101	Intro to CS	Dr. Layla	A
102	Sami	CS101	Intro to CS	Dr. Layla	A-
102	Sami	PHY301	Quantum Physics	Dr. Amani	C

2.3 Update Anomaly

Problem: Redundant data leads to inconsistencies during updates.

Dr. Layla teaches `CS101`. Her name appears in **every row** where `CS101` is offered. If Dr. Layla changes her name (e.g., to *Prof. Layla*), you must update every row with `CS101`. If you miss one row, data becomes inconsistent.

StudentID	StudentName	CourseID	CourseTitle	Instructor	Grade
101	Ahmed	CS101	Intro to CS	**Prof. Layla**	A
101	Ahmed	MATH201	Calculus II	Dr. Ibrahem	B+
102	Sami	CS101	Intro to CS	**Dr. Layla** ← inconsistent!	A-
102	Sami	PHY301	Quantum Physics	Dr. Amani	C

2.4 Solution: Normalization

Normalization is the process of decomposing a table into smaller, well-structured tables to:

- **Eliminate redundancy** — the same data isn't stored multiple times unnecessarily
- **Avoid anomalies** — relationships between tables are clear and logical
- **Ensure data integrity** — queries run faster and are easier to maintain

We split the `Enrollment` table into three tables:

Table 27: *Students*

Table 28: *Courses*

Table 29: *Enrollments*

StudentID	StudentName
101	Ahmed
102	Sami

CourseID	CourseTitle	Instru
CS101	Intro to CS	Dr. Layla
MATH201	Calculus II	Dr. Ibrahem
PHY301	Quantum Physics	Dr. Amani

StudentID	CourseID	Grad
101	CS101	A
101	MATH201	B+
102	CS101	A-
102	PHY301	C

Now each anomaly disappears:

- **Adding a new course?** Just insert into `Courses` — no student needed.
- **A student drops a course?** Delete only from `Enrollments` — course data stays intact.
- **Instructor name changes?** Update one row in `Courses` — done.

> **Note**
>
> When you perform **ER-to-Relational Mapping**, following standard mapping rules automatically structures your tables to be in **at least 3NF** in the vast majority of cases:
>
> - **Entities → Tables:** Each entity type becomes a separate relation, giving entity attributes a clear home around a chosen entity key.
> - **Relationships → Foreign Keys or Junction Tables:** Binary 1:N relationships use foreign keys, and M:N relationships become separate junction tables (e.g., `Enrollments`), which naturally isolates composite key attributes like `Grade`.
> - **Attributes → Atomic Fields:** Multi-valued attributes are broken out into separate tables, satisfying 1NF.

3 Functional Dependencies

Before we can normalize, we need a precise language for describing what determines what in a relation. That language is **Functional Dependencies (FDs)**.

In database design, FDs provide the formal mathematical foundation used to capture real-world business rules about how attributes relate to one another.

Think of an FD like a deterministic mathematical function:

$$y = f(x) \implies \text{knowing } x \text{ always gives you exactly one } y$$

The database equivalent is written $X \to Y$ and reads:

> **"For every valid instance of X, there is a unique value of Y in the table."**

In other words, for any two tuples (rows) t_1 and t_2 in a relation state $r(R)$:

$$\text{if } t_1[X] = t_2[X], \text{ then } t_1[Y] = t_2[Y]$$

If two rows share the same X value, they **must** share the same Y value—no exceptions.

> 📖 DEFINITION 1: **Functional Dependency**
>
> A Functional Dependency X→Y (read as "X functionally determines Y") holds in a relation R if, for **any** valid state of R, whenever two tuples agree on the attributes of X, they **must** also agree on the attributes of Y.
>
> - **Determinant:** The left-hand side (X).
> - **Dependent:** The right-hand side (Y).

3.1 Why FDs Are Essential

Without functional dependencies, you cannot formally define keys or prove that a design is free of data anomalies. FDs drive the entire theoretical foundation of relational database design:

1. **Functional Dependencies ($X \to Y$):** Formally define attribute-level constraints based on domain semantics (e.g., $\text{StudentID} \to \text{StudentName}$).
2. **Keys (Super & Candidate Keys):** Derived directly from FDs by computing attribute set closures (X^+). A set X is a superkey if $X^+ = R$, and a candidate key if X is also minimal.
3. **Normal Forms ($1\text{NF} \to 2\text{NF} \to 3\text{NF} \to \text{BCNF}$):** Evaluated strictly by testing conditions on FDs and keys:
   - **2NF:** Eliminates **partial dependencies** ($\text{Non-Prime} \subset \text{Candidate Key}$).
   - **3NF:** Eliminates **transitive dependencies** ($\text{Non-Prime} \to \text{Non-Prime}$).
   - **BCNF:** Ensures that for every non-trivial FD $X \to Y$, X must be a **superkey**.

> 📝 EXAMPLE 1: **Attribute-Level Determinism**
>
> A company assigns each employee a unique employee number (EmpNo) or Social Security Number (SSN).
>
> $$\text{SSN} \to \text{Name}$$
>
> - **Meaning:** If you know the value of SSN, you can uniquely identify the Name of that employee.

- **Reading the Notation:** The expression $\text{SSN} \rightarrow \text{Name}$ is read as:

  > "SSN *determines* Name" or "SSN *functionally implies* Name."

Does the Reverse Hold ($\text{Name} \rightarrow \text{SSN}$)?

$$\text{Name} \rightarrow \text{SSN} \quad \times (\text{False})$$

No. It would violate database consistency if two distinct employees happened to share the same name (e.g., two employees named "Ahmed"). Because their SSN values would be different, Name cannot uniquely determine SSN.

> ⓘ **FDs Come from Domain Knowledge, Not Data Snapshots**
>
> Functional dependencies are constraints on the **relation schema** itself, derived from real-world business rules—**not** from a temporary snapshot of data:
>
> - × **Do not infer an FD** just because it happens to hold in the current rows of a table.
> - ✓ **An FD is valid** only if it holds for *every possible valid state* the database could ever enter.

3.2 Discovering FDs from Stakeholders

Functional dependencies are not invented by database designers; they are discovered by interviewing domain experts (stakeholders) who own and understand the real-world business rules.

📝 EXAMPLE 2: **Stakeholder Interview & FD Extraction**

By asking targeted questions to the **University Registrar**, we extract the core business constraints and translate them directly into mathematical Functional Dependencies:

Stakeholder Question	Registrar's Business Rule	Extracted FD Notation
"Can a student ID belong to multiple students?"	*"No, each student has a unique ID and a single official name."*	$\text{StudentID} \rightarrow \text{StudentName}$
"Can a course ID have different course titles?"	*"No, a course code like CS101 always has a single fixed title."*	$\text{CourseID} \rightarrow \text{CourseTitle}$
"Can multiple instructors teach the same course?"	*"Yes, but in any given term, each course is taught by only one instructor."*	$(\text{CourseID}, \text{Term}) \rightarrow \text{Instructor}$
"What determines the final grade assigned to a student?"	*"A grade is assigned to a specific student for a specific course."*	$(\text{StudentID}, \text{CourseID}) \rightarrow \text{Grade}$

Consider the following sample table containing student enrollment records:

StudentID	StudentName	CourseID	CourseTitle	Term	Instructor	Grade
101	Ahmed	CS101	Intro to CS	Fall 2026	Dr. Layla	A
101	Ahmed	MATH201	Calculus II	Fall 2026	Dr. Ibrahem	B+
102	Sami	CS101	Intro to CS	Fall 2026	Dr. Layla	A-
102	Sami	PHY301	Quantum Physics	Spring 2026	Dr. Amani	C
103	Tariq	CS101	Intro to CS	Spring 2026	Prof. Omar	B

- **Data is Just an Instance:** The table rows above illustrate the rules, but the rules (FDs) are defined by domain logic, not the data snapshot.
- **Formal Validation:**
  - $\text{StudentID} \rightarrow \text{StudentName}$ holds because whenever two rows share the same StudentID (e.g., 101), they share the exact same StudentName ("Ahmed").
  - $(\text{CourseID}, \text{Term}) \rightarrow \text{Instructor}$ holds because $(\text{CS101}, \text{Fall 2026})$ uniquely determines "Dr. Layla", while $(\text{CS101}, \text{Spring 2026})$ uniquely determines "Prof. Omar".

3.2.1 FD Validation

An Functional Dependency (FD) $X \to Y$ is **valid** (holds) in a relation state $r(R)$ if and only if for every pair of tuples (rows) t_1 and t_2 in $r(R)$ that agree on the attribute set X, they must also agree on the attribute set Y.

Formally:

$$X \to Y \iff \forall t_1, t_2 \in r(R), \quad \text{if } t_1[X] = t_2[X], \text{ then } t_1[Y] = t_2[Y]$$

📝 EXAMPLE 3: **Validating FDs on a Relation Instance**

Consider the relation instance $R(A, B, C, D)$:

A	B	C	D
1	1	2	3
1	2	2	3
1	3	2	3
2	4	5	6
5	6	7	8

Which of the following FDs are satisfied by this relation?

1. $A \to B$
2. $A \to CD$
3. $AB \to CD$
4. $C \to D$
5. $B \to A$
6. $BD \to AC$
7. $AD \to BC$
8. $D \to B$
9. $D \to C$
10. $C \to A$

✅ Show Solution

1. $A \to B$

   $\times$ Does not hold. For $A = 1$, there are three distinct values of B $(1, 2, 3)$.

2. $A \to CD$

✓ Holds. For each value of A, there is a unique (C, D) pair:

- $A = 1 \implies (2, 3)$
- $A = 2 \implies (5, 6)$
- $A = 5 \implies (7, 8)$

3. $AB \to CD$

   ✓ Holds. Every (A, B) combination is distinct across all rows, so each naturally maps to a unique (C, D) pair.

4. $C \to D$

   ✓ Holds.

   - $C = 2 \implies D = 3$
   - $C = 5 \implies D = 6$
   - $C = 7 \implies D = 8$

5. $B \to A$

   ✓ Holds. Every value of B is unique in this instance, mapping to a single A value.

6. $BD \to AC$

   ✓ Holds. Each (B, D) pair appears at most once in the table.

7. $AD \to BC$

   × Does not hold. For $(A, D) = (1, 3)$, there are three distinct (B, C) pairs: $(1, 2)$, $(2, 2)$, and $(3, 2)$.

8. $D \to B$

   × Does not hold. For $D = 3$, B takes values $1, 2$, and 3.

9. $D \to C$

   ✓ Holds.

   - $D = 3 \implies C = 2$
   - $D = 6 \implies C = 5$
   - $D = 8 \implies C = 7$

10. $C \to A$

    ✓ Holds. Each C value maps to a unique A value ($2 \implies 1$, $5 \implies 2$, $7 \implies 5$).

> ⚠️ **Important Distinction:**
>
> While dependencies like $B \to A$ and $C \to A$ hold in this *particular sample instance*, they cannot be declared general schema FDs unless supported by true real-world business rules. A single counterexample disproves an FD, but an instance satisfying an FD does not prove it holds universally for all future data.

3.3 Keys, Prime, and Non-Prime Attributes

3.3.1 Review of Keys

📖 **DEFINITION 2: Key Attribute**

An attribute that uniquely identifies each entity instance.

📖 **DEFINITION 3: Super Key**

A set of one or more attributes (columns) that uniquely identifies a row (tuple) within a table. Any combination of columns that prevents duplicate rows is a super key.

📖 **DEFINITION 4: Candidate Key**

A **Candidate Key** is a *minimal* Super Key. It has the exact same unique identification property as a Super Key, but it contains **no redundant attributes**. If you remove any attribute from a Candidate Key, it ceases to be a key.

📝 **EXAMPLE 4: Candidate Keys vs. Super Keys**

Suppose we have a relation $R(\text{SSN}, \text{Name}, \text{Address}, \text{Phone})$ with the functional dependency:

$$\text{SSN} \rightarrow \{\text{Name}, \text{Address}, \text{Phone}\}$$

Key Classifications

1. **Candidate Key:** $\{\text{SSN}\}$
   - **Closure:** $\{\text{SSN}\}^+ = \{\text{SSN}, \text{Name}, \text{Address}, \text{Phone}\} = R$ (it uniquely identifies all attributes in the relation, so it is a **superkey**).
   - **Minimality:** No proper subset of $\{\text{SSN}\}$ can uniquely identify the entire relation. Therefore, $\{\text{SSN}\}$ is a **candidate key**.
2. **Superkey (Not a Candidate Key):** $\{\text{SSN}, \text{Name}\}$
   - **Closure:** $\{\text{SSN}, \text{Name}\}^+ = \{\text{SSN}, \text{Name}, \text{Address}, \text{Phone}\} = R$ (it uniquely identifies all attributes, so it is a **superkey**).
   - **Minimality Check:** It contains a proper subset, $\{\text{SSN}\}$, which is already a superkey on its own. Because Name is redundant for uniqueness, $\{\text{SSN}, \text{Name}\}$ fails the minimality requirement and is **not a candidate key**.

Core Concept Breakdown

$$\text{Candidate Key} = \text{Superkey} + \text{Minimality}$$

- **Superkey:** Any attribute set X whose closure covers the entire relation schema ($X^+ = R$).
- **Candidate Key:** A minimal superkey—removing any attribute from X causes $X^+ \neq R$.

3.3.2 Prime vs. Non-Prime Attributes

This distinction is critical for 2NF and 3NF normalization.

- **Prime Attribute:** An attribute that is a member of **ANY** Candidate Key.
- **Non-Prime Attribute:** An attribute that is **NOT** a member of any Candidate Key.

📝 EXAMPLE 5: **Candidate Keys & Attribute Classification**

For a relation $R(A, B, C, D)$ with candidate keys $CK_1 = \{A, B\}$ and $CK_2 = \{B, C\}$:

- **Prime Attributes:** $\{A, B, C\}$

  *An attribute is prime if it is a member of **at least one** candidate key.*

  - $A \in CK_1$
  - $B \in CK_1$ and $B \in CK_2$
  - $C \in CK_2$
- **Non-Prime Attribute:** $\{D\}$

  *An attribute is non-prime if it does **not** belong to any candidate key.*

3.4 Deriving Keys and Functional Dependencies

In database design, we are rarely handed all functional dependencies in their final form. We typically start with a set of known business rules F and must derive implied dependencies to analyze schema properties, verify rules, and discover candidate keys.

3.4.1 Armstrong's Axioms (The Inference Rules)

To derive new, sound, and complete functional dependencies from a given set F, we use **Armstrong's Axioms**. The set of all possible FDs that can be logically inferred from F is called the **closure of** F, denoted as F^+.

3.4.1.1 Primary Axioms

- **Reflexivity:** If $Y \subseteq X$, then $X \to Y$.

(A set of attributes always determines any of its subsets.)

- **Augmentation:** If $X \to Y$, then $XZ \to YZ$ for any attribute set Z.

  (Adding the same attributes to both sides preserves the dependency.)

- **Transitivity:** If $X \to Y$ and $Y \to Z$, then $X \to Z$.

  (Functional determination propagates across intermediate attributes.)

📝 EXAMPLE 6:

Imagine a university table tracking student enrollments with columns: `StudentID`, `CourseID`, and `Semester`.

1. **Given Set (X):**
   - Let $X = \{\text{StudentID}, \text{CourseID}, \text{Semester}\}$
2. **Choose a Subset ($Y \subseteq X$):**
   - Let $Y = \{\text{StudentID}\}$
3. **Result ($X \to Y$):**
   - Since Y is a subset of X, we get:

     $$\{\text{StudentID}, \text{CourseID}, \text{Semester}\} \to \{\text{StudentID}\}$$

Intuition: If you already know the specific combination of a student, course, and semester, you inherently know the student's ID. Knowing the whole collection of attributes guarantees you know any individual part (or combination of parts) within that collection.

📝 EXAMPLE 7:

1. **Given Functional Dependency ($X \to Y$):**
   - Knowing a product's barcode (X) determines its unit price (Y).
   - $\text{Barcode} \to \text{Price}$
2. **Add Attribute Z to Both Sides ($XZ \to YZ$):**
   - Let $Z = \{\text{StoreLocation}\}$.
   - Augmenting both sides with the store location gives:

     $$\{\text{Barcode}, \text{StoreLocation}\} \to \{\text{Price}, \text{StoreLocation}\}$$

Intuition: If a barcode determines a price, then knowing the barcode *and* the store location will determine the price *and* that same store location.

📝 EXAMPLE 8:

Imagine a corporate database tracking employee assignments with columns: `EmployeeID`, `DeptID`, and `BuildingNo`.

1. **Given Functional Dependencies:**
   - Knowing an employee's ID determines their department ID ($X \to Y$):

     $$\text{EmployeeID} \to \text{DeptID}$$

   - Knowing the department ID determines the building where that department is located ($Y \to Z$):

     $$\text{DeptID} \to \text{BuildingNo}$$

2. **Result ($X \to Z$):**
   - By transitivity, we can chain these together to conclude that knowing the employee's ID indirectly determines their building:

     $$\text{EmployeeID} \to \text{BuildingNo}$$

Intuition: If X leads to Y, and Y leads to Z, then X transitively leads to Z because Y acts as a bridge.

3.4.1.2 Secondary (Derived) Rules

From the three primary axioms, we derive practical helper rules:

- **Union (Additive):** If $X \to Y$ and $X \to Z$, then $X \to YZ$.
- **Decomposition:** If $X \to YZ$, then $X \to Y$ and $X \to Z$.
- **Pseudotransitivity:** If $X \to Y$ and $WY \to Z$, then $WX \to Z$.

📝 EXAMPLE 9: **Union (Additive) Rule**

Given Functional Dependencies:

- Knowing an `EmployeeID` determines their `Name`:

  $$\text{EmployeeID} \to \text{Name}$$

- Knowing the same `EmployeeID` determines their `Email`:

  $$\text{EmployeeID} \to \text{Email}$$

- **Result ($X \to YZ$):**

- By the Union rule, knowing the `EmployeeID` allows you to determine both together:

$$\text{EmployeeID} \rightarrow \{\text{Name}, \text{Email}\}$$

Intuition: If one key can look up your name and the same key can look up your email, you can group them into a single lookup for both pieces of information.

📝 EXAMPLE 10: **Pseudotransitivity Rule**

Given Functional Dependencies:

- Knowing a `StudentID` (X) determines their current `Major` (Y):

$$\text{StudentID} \rightarrow \text{Major}$$

- Knowing a `Major` (Y) *combined with* a specific `CourseLevel` (W) determines the maximum `AllowedCredits` (Z):

$$\{\text{Major}, \text{CourseLevel}\} \rightarrow \text{AllowedCredits}$$

- **Result ($WX \rightarrow Z$):**
  - By Pseudotransitivity, we can substitute `Major` with `StudentID`, meaning knowing the student's ID *and* the course level gives you the allowed credits:

$$\{\text{StudentID}, \text{CourseLevel}\} \rightarrow \text{AllowedCredits}$$

Intuition: Since X provides Y, you can feed X into the second rule wherever Y is required, bridging X and W together to reach Z.

3.4.2 Practical Tips for Spotting Keys Quickly

Before running any algorithm, experienced database designers use these **visual heuristics** to immediately narrow down the search for candidate keys. These tips come from analyzing the left-hand side (LHS) and right-hand side (RHS) of the given FDs.

💡 **Tip 1: The "Must-Have" Rule (Never on RHS)**

Any attribute that **never appears on the right-hand side** of *any* FD **must** be part of **every** Candidate Key. *Why?* Because there is no way to derive this attribute from any other attribute. The only way it can appear in a closure is if you include it in the starting set.

Tip 2: The "Never a Key" Rule (Only on RHS)

Any attribute that **only appears on the right-hand side** (and never on the left-hand side) of any FD can **never** be part of any Candidate Key. *Why?* Because this attribute is always "determined by" something else. It adds no determining power. Including it in a key would make the key non-minimal.

Tip 3: The "Both Sides" Rule

Attributes that appear on **both** the LHS and RHS of various FDs are the "candidate" attributes. They *might* be part of a candidate key, depending on closure computation.

Tip 4: Single-Attribute Quick Check

Before trying combinations, always compute the closure of each individual "Must-Have" attribute and each "Both Sides" attribute. If any single attribute's closure equals R, that attribute alone is a Candidate Key.

3.4.3 The Attribute Closure Algorithm

Computing the full set of all dependencies F^+ can be exponentially large. To determine whether a specific set of attributes X can uniquely identify the entire table—or to test if a specific FD $X \to Y$ holds—we compute the **Attribute Closure** of X, denoted as X^+.

📖 DEFINITION 5: **The attribute closure**

The attribute closure X^+ is the set of all attributes in R that are functionally determined by X under the given set of dependencies F.

Algorithm: Computing Attribute Closure

1. **Initialization:** Set $X^+ = X$.
2. **Iterative Expansion:** Repeatedly search F for an FD $A \to B$ such that $A \subseteq X^+$ and $B \nsubseteq X^+$. If found, set $X^+ = X^+ \cup B$.
3. **Termination:** Repeat Step 2 until a full pass over F yields no new attributes (a fixpoint is reached).

📝 EXAMPLE 11: **Basic Step-by-Step Expansion**

Given:

Relation Schema: $R(A, B, C, D, E, F)$

Functional Dependency Set F:

1. $A \to B$
2. $\{B, C\} \to D$
3. $D \to E$
4. $E \to F$

Compute the attribute closure of $\{A, C\}$, denoted as $\{A, C\}^+$.

✅ Show Solution

- **Initialization:**

$$\{A, C\}^+ = \{A, C\}$$

- **Iteration 1:**
  - Check $A \to B$: LHS $A \subseteq \{A, C\}^+$. Add B.

$$\{A, C\}^+ = \{A, B, C\}$$

  - Check $\{B, C\} \to D$: LHS $\{B, C\} \subseteq \{A, B, C\}^+$. Add D.

$$\{A, C\}^+ = \{A, B, C, D\}$$

  - Check $D \to E$: LHS $D \subseteq \{A, B, C, D\}^+$. Add E.

$$\{A, C\}^+ = \{A, B, C, D, E\}$$

  - Check $E \to F$: LHS $E \subseteq \{A, B, C, D, E\}^+$. Add F.

$$\{A, C\}^+ = \{A, B, C, D, E, F\}$$

Termination:

$\{A, C\}^+ = \{A, B, C, D, E, F\} = R$.

> **Conclusion:** Since $\{A, C\}^+$ contains all attributes of R, $\{A, C\}$ is a **superkey**. Because neither $A^+ = \{A, B\}$ nor $C^+ = \{C\}$ covers R, $\{A, C\}$ is also a **candidate key**.

📝 EXAMPLE 12: **Partial Closure & Unused Dependencies**

Given:

Relation Schema: $R(A, B, C, D, E)$

Functional Dependency Set F:

1. $A \to B$
2. $B \to C$
3. $\{C, D\} \to E$
4. $E \to A$

Compute:

- The attribute closure of $\{A, D\}$, denoted as $\{A, D\}^+$.
- The attribute closure of $\{A, C\}$, denoted as $\{A, C\}^+$.

✅ Show Solution

Task: Compute the attribute closure of $\{A, D\}$, denoted as $\{A, D\}^+$.

Trace:

- **Initialization:**

$$\{A, D\}^+ = \{A, D\}$$

- **Iteration 1:**
  - Check $A \to B$: LHS $A \subseteq \{A, D\}^+$. Add B.

$$\{A, D\}^+ = \{A, B, D\}$$

  - Check $B \to C$: LHS $B \subseteq \{A, B, D\}^+$. Add C.

$$\{A, D\}^+ = \{A, B, C, D\}$$

  - Check $\{C, D\} \to E$: LHS $\{C, D\} \subseteq \{A, B, C, D\}^+$. Add E.

$$\{A, D\}^+ = \{A, B, C, D, E\}$$

- **Termination:**

$\{A, D\}^+ = \{A, B, C, D, E\} = R$.

Task: Compute the attribute closure of $\{A, C\}$, denoted as $\{A, C\}^+$.

Trace:

- **Initialization:**

$$\{A, C\}^+ = \{A, C\}$$

- **Iteration 1:**
  - Check $A \to B$: LHS $A \subseteq \{A, C\}^+$. Add B.

$$\{A, C\}^+ = \{A, B, C\}$$

  - Check $B \to C$: C is already in the closure. No change.
  - Check $\{C, D\} \to E$: LHS $\{C, D\} \not\subseteq \{A, B, C\}^+$ (missing D). Cannot apply.
  - Check $E \to A$: LHS $E \not\subseteq \{A, B, C\}^+$. Cannot apply.
- **Iteration 2:**

No new attributes can be added. Pass completes without changes.

- **Termination:**

$$\{A, C\}^+ = \{A, B, C\} \neq R$$

> **Conclusion:** $\{A, C\}$ cannot determine D or E, so it is **not** a superkey.

3.4.4 The Complete Key-Finding Algorithm

This is the systematic procedure to find all Candidate Keys, classify Prime and Non-Prime attributes, and enumerate all Super Keys.

Step 1 — Classify all attributes into three groups:

- **Group M (Must-Have):** Attributes that never appear on the RHS of any FD. These must belong to every Candidate Key.
- **Group N (Never-a-Key):** Attributes that only appear on the RHS (never on the LHS). These can never be part of any Candidate Key.
- **Group B (Both-Sides):** Attributes that appear on both LHS and RHS across different FDs.

Step 2 — Compute the closure of M:

- Compute M^+ using the attribute closure algorithm.
- If $M^+ = R$, then M is the only Candidate Key. **Stop.**
- If $M^+ \neq R$, proceed to Step 3.

Step 3 — Find Candidate Keys by extending M:

1. Systematically add attributes from Group B to M, one at a time, then in pairs, and so on.
2. For each combination, compute the attribute closure.
3. If the closure equals R (i.e., $X^+ = R$), you have found a **Super Key**.
4. **Check Minimality:** Test each attribute in your Super Key by trying to remove it. If removing any attribute still results in a closure of R, then the attribute was redundant, meaning the set is *not* minimal and therefore **not a Candidate Key**.
5. *Never* add attributes from Group N, as they cannot help form a minimal key.

Step 4 — Identify Prime and Non-Prime attributes:

- **Prime Attribute:** Any attribute that appears in *at least one* Candidate Key.

- **Non-Prime Attribute:** Any attribute that appears in *no* Candidate Key.

Step 5 — Enumerate all Super Keys (if needed):

- Every Candidate Key is automatically a Super Key.
- Every superset of a Candidate Key is also a Super Key.
- **The Complete Set of Super Keys:** All possible unions of Candidate Keys combined with any combination of Non-Prime attributes (along with combinations with each other).

📝 EXAMPLE 13: **Full Worked Example**

Relation $R(A, B, C, D, E)$ with the set of functional dependencies:

$$F = \{A \to B,\ B \to A,\ AC \to D,\ D \to E\}$$

Find all Candidate Keys, classify Prime and Non-Prime attributes, and list all Super Keys.

✅ Show Solution

Step 1: Classify Attributes

First, analyze where each attribute appears across the left-hand side (LHS) and right-hand side (RHS) of the FDs:

Attribute	Appears on LHS?	Appears on RHS?	Group
A	Yes ($A \to B$)	Yes ($B \to A$)	**Group B** (Both-Sides)
B	Yes ($B \to A$)	Yes ($A \to B$)	**Group B** (Both-Sides)
C	Yes ($AC \to D$)	**No**	**Group M** (Must-Have)
D	Yes ($D \to E$)	Yes ($AC \to D$)	**Group B** (Both-Sides)
E	**No**	Yes ($D \to E$)	**Group N** (Never-a-Key)

Immediate Conclusions from Heuristics:

1. C must be in every Candidate Key (it never appears on the RHS).
2. E can never be in any Candidate Key (it only appears on the RHS).
3. We only need to test combinations of C with attributes from Group B: $\{A, B, D\}$.

Step 2: Compute the Closure of $M = \{C\}$

Iteration	Current X+	FD Applied	New X+
Start	$\{C\}$	—	$\{C\}$
1	$\{C\}$	No FD has LHS $\subseteq \{C\}$	$\{C\}$

- **Result:** $\{C\}^+ = \{C\} \neq R$. Since the closure does not cover the entire relation, we must extend M with attributes from Group B.

Step 3: Extend M and Find Candidate Keys

We systematically test adding attributes from Group B ($\{A, B, D\}$) to our core set C.

Try $\{A, C\}$:

Iteration	Current X+	FD Applied	New X+
Start	$\{A, C\}$	—	$\{A, C\}$
1	$\{A, C\}$	$A \to B$	$\{A, B, C\}$
2	$\{A, B, C\}$	$AC \to D$ (since $A, C \in X^+$)	$\{A, B, C, D\}$
3	$\{A, B, C, D\}$	$D \to E$	$\{A, B, C, D, E\}$

- **Result:** $\{A, C\}^+ = \{A, B, C, D, E\} = R$. ✅ **Super Key found!**
- **Minimality Check:**
  - Remove A: $\{C\}^+ = \{C\} \neq R$ ❌ (Cannot remove A)
  - Remove C: $\{A\}^+ = \{A, B\} \neq R$ ❌ (Cannot remove C)
- ✅ Therefore, $\{A, C\}$ **is a Candidate Key.**

Try $\{B, C\}$:

Iteration	Current X+	FD Applied	New X+
Start	$\{B, C\}$	—	$\{B, C\}$
1	$\{B, C\}$	$B \to A$	$\{A, B, C\}$
2	$\{A, B, C\}$	$AC \to D$	$\{A, B, C, D\}$
3	$\{A, B, C, D\}$	$D \to E$	$\{A, B, C, D, E\}$

- **Result:** $\{B, C\}^+ = R$. ✅ **Super Key found!**
- **Minimality Check:**
  - Remove B: $\{C\}^+ = \{C\} \neq R$ ❌
  - Remove C: $\{B\}^+ = \{B, A\} \neq R$ ❌
- ✅ Therefore, $\{B, C\}$ **is a Candidate Key.**

Try $\{D, C\}$:

Iteration	Current X+	FD Applied	New X+
Start	$\{D, C\}$	—	$\{D, C\}$
1	$\{D, C\}$	$D \to E$	$\{C, D, E\}$
2	$\{C, D, E\}$	No FD has LHS $\subseteq \{C, D, E\}$ producing A or B	$\{C, D, E\}$

- **Result:** $\{D, C\}^+ = \{C, D, E\} \neq R$. ❌ **Not a Super Key.**

(Note: We can skip larger pairs like $\{A, D, C\}$ because they would contain the valid Candidate Key $\{A, C\}$ as a subset, making them non-minimal).

Step 4: Classify Prime and Non-Prime Attributes

Attribute	In CK {A,C}?	In CK {B,C}?	Classification
A	✅ Yes	—	**Prime**
B	—	✅ Yes	**Prime**
C	✅ Yes	✅ Yes	**Prime**
D	❌ No	❌ No	**Non-Prime**
E	❌ No	❌ No	**Non-Prime**

- **Prime Attributes:** $\{A, B, C\}$
- **Non-Prime Attributes:** $\{D, E\}$

Step 5: Enumerate All Super Keys

Every Candidate Key is a Super Key, and every superset of a Candidate Key is also a Super Key. To find all Super Keys, we take our Candidate Keys ($\{A, C\}$ and $\{B, C\}$) and append **all possible combinations of the Non-Prime attributes** ($\{D, E\}$):

1. **Supersets of $\{A, C\}$:**
   - $\{A, C\}$ *(Candidate Key)*
   - $\{A, B, C\}$ *(Added B)*
   - $\{A, C, D\}$ *(Added non-prime D)*
   - $\{A, C, E\}$ *(Added non-prime E)*
   - $\{A, B, C, D\}$ *(Added B, D)*
   - $\{A, B, C, E\}$ *(Added B, E)*
   - $\{A, C, D, E\}$ *(Added D, E)*
   - $\{A, B, C, D, E\}$ *(Added all)*
2. **Supersets of $\{B, C\}$ (Unique additions):**
   - $\{B, C\}$ *(Candidate Key)*
   - $\{B, C, D\}$ *(Added non-prime D)*
   - $\{B, C, E\}$ *(Added non-prime E)*
   - $\{B, C, D, E\}$ *(Added D, E)*

- **Complete Unique Set of All Super Keys (12 total):**

$$SK = \{A, C\}, \{B, C\}, \{A, B, C\}, \{A, C, D\}, \{A, C, E\}, \{B, C, D\}, \{B, C, E\}, \{A, B, C, D\}, \{A,$$

💡 **The Pattern:** Notice that every Super Key contains either $\{A, C\}$ or $\{B, C\}$ as a subset. The Non-Prime attributes $\{D, E\}$ act as "passengers"—appending them to any Candidate Key creates a valid Super Key, but they

contribute no determining power of their own.

3.5 Types of Functional Dependencies

Not all functional dependencies are created equal. Some represent structural flaws (anomalies) in the database design, while others are mathematically trivial. Understanding these types is the exact mechanism used to define **Second Normal Form (2NF)** and **Third Normal Form (3NF)**.

Table 30: *Types of FDs*

Term	Mathematical Definition	Meaning & Impact
Trivial FD	$X \to Y$ where $Y \subseteq X$	Always true by definition. Carries no new information.
Non-trivial FD	$X \to Y$ where $Y \not\subseteq X$	The meaningful FDs that define actual business rules.
Full FD	$X \to Y$ and no proper subset of X determines Y	The ideal state. Y depends on the *whole* key.
Partial FD	$X \to Y$ and a proper subset of X determines Y	**Red flag for 2NF.** Y depends on only *part* of a composite key.
Transitive FD	$X \to Y \to Z$ (where $Y \not\to X$)	**Red flag for 3NF.** Z depends on a non-key attribute.

3.5.1 Trivial Functional Dependency

📖 DEFINITION 6: **Trivial Functional Dependency**

An FD $X \to Y$ is trivial if Y is a subset of X ($Y \subseteq X$).

📝 EXAMPLE 14:

Consider an employee table where we know both the employee's ID and their name.

- **FD:** $\{\text{EmpID}, \text{Name}\} \to \text{EmpID}$ (Since EmpID is part of the left-hand side set, this is trivial).

EmpID	Name	Department
101	Ahmed	Computer Science
102	Mahmoud	Engineering

If you already know that an employee has ID `101` and is named **Ahmed**, stating that $\{\text{EmpID}, \text{Name}\}$ determines EmpID tells you nothing new. You already had the ID.

3.5.2 Non-Trivial Functional Dependency

📖 DEFINITION 7: **Non-Trivial Functional Dependency**

An FD $X \rightarrow Y$ is non-trivial if Y is not a subset of X ($Y \not\subseteq X$). *If X and Y share no common attributes at all, i.e., $X \cap Y = \emptyset$, it is called **completely non-trivial**).*

📝 EXAMPLE 15:

Consider an employee table:

- **FD:** $\text{EmpID} \rightarrow \text{Department}$ (Knowing the employee ID uniquely determines their department).

EmpID	Name	Department
101	Ahmed	Computer Science
102	Mahmoud	Engineering

The attribute on the right (Department) is completely separate from the left (EmpID). This is a genuine business rule used to design the database.

3.5.3 Full Functional Dependency

📖 DEFINITION 8: **Full Functional Dependency**

An FD $X \rightarrow Y$ is a full functional dependency if it is non-trivial, and removing *any* attribute A from X breaks the dependency (meaning no proper subset of X determines Y).

📝 EXAMPLE 16:

Imagine a university enrollment relation where a student takes a course across a specific semester. The Primary Key is a composite key: $\{\text{StudentID}, \text{CourseID}\}$.

- **FD:** $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{Grade}$

StudentID	CourseID	Grade
201 (Ahmed)	CS101	A
201 (Ahmed)	MATH202	B
202 (Sara)	CS101	A

Can you know **Ahmed**'s Grade by looking at StudentID alone? No, because Ahmed takes multiple courses with different grades. Can you know the Grade by looking at CourseID alone? No, because many students take CS101 and get different grades. **Grade** requires *both* attributes together to be uniquely determined.

3.5.4 Partial Functional Dependency

📖 DEFINITION 9: **Partial Functional Dependency**

An FD $X \rightarrow Y$ is a partial dependency if a *proper subset* of X can already determine Y.

📝 EXAMPLE 17:

Using the same enrollment table, suppose we also store the student's name:

- **FD:** $\{\text{StudentID}, \text{CourseID}\} \rightarrow \text{StudentName}$

StudentID	CourseID	StudentName
201	CS101	Ahmed
201	MATH202	Ahmed
202	CS101	Sara

- **The Red Flag:** StudentName depends *only* on StudentID, which is just a proper subset of the composite key $\{\text{StudentID}, \text{CourseID}\}$.
- **The Problem:** If **Ahmed** changes his name, you have to update it across multiple rows. If you miss one, your database becomes inconsistent. This violates **2NF**, and the fix is to move StudentName into a separate table where StudentID is the sole primary key.

3.5.5 Transitive Functional Dependency

📖 DEFINITION 10: **Transitive Functional Dependency**

An FD $X \to Z$ is transitive if there is an intermediate attribute set Y such that $X \to Y$ and $Y \to Z$ (where $Y \not\to X$, and Y is not a candidate key).

📝 EXAMPLE 18:

Consider an employee table where EmpID is the primary key:

- **FD 1:** $\mathrm{EmpID} \to \mathrm{DeptID}$ (An employee belongs to a department ID)
- **FD 2:** $\mathrm{DeptID} \to \mathrm{DeptName}$ (A department ID determines the department name)
- **Transitive FD:** $\mathrm{EmpID} \to \mathrm{DeptName}$ (Through DeptID)

EmpID	Name	DeptID	DeptName
101	Ahmed	D01	IT
102	Khaled	D02	HR
103	Yousef	D01	IT

- **The Red Flag:** $\mathrm{DeptName}$ is a non-prime attribute, but it depends on DeptID, which is *also* a non-prime attribute (just a regular column, not a candidate key for this table).
- **The Problem:** If Department `D01` changes its name from **IT** to **Computer Science**, you have to update every single employee belonging to that department. This violates **3NF**, and the fix is to extract department data into its own table ($\mathrm{DEPARTMENT}$).

4 Normalization

Normalization is the systematic approach of effectively designing a database so that it minimizes data redundancy and inconsistency. It is generally done through a series of steps called **Normal Forms (NF)**.

Each normal form builds on the previous one—a relation in a higher NF automatically satisfies all lower NFs:

$$1NF \subset 2NF \subset 3NF \subset BCNF \subset 4NF \subset 5NF$$

Normal Form	Key Requirement	What it Eliminates
1NF	All attribute values are atomic	Multi-valued / repeating groups
2NF	1NF + no partial FDs on any candidate key	Redundancy from partial dependencies
3NF	2NF + no transitive FDs (with prime-attr exception)	Transitive dependencies via non-key attributes
BCNF	Every FD determinant is a superkey	All non-trivial FD-based redundancy
4NF	BCNF + no non-trivial MVDs	Multi-valued dependency redundancy
5NF	4NF + no non-trivial join dependencies	Decomposition artifacts

4.1 First Normal Form (1NF)

A relation is in 1NF if:

- Every attribute contains only **atomic** (single-valued, indivisible) values
- There are no repeating groups or multi-valued attributes
- There is a **primary key** that uniquely identifies each row

📝 EXAMPLE 19: **Violating 1NF**

Consider an unnormalized student table where a student can enroll in multiple courses, and those courses are stored together inside a single comma-separated cell:

StudentNo	StudentName	Courses
S1	Ahmed	DS, Algorithms
S2	Hussein	Databases

✅ Show Solution

The Issue: The `Courses` attribute is **not atomic** because it holds multiple values in a single cell. This directly violates 1NF and prevents standard relational querying (like indexing or joining on individual course identifiers).

To bring the relation into 1NF, we split the multi-valued field into separate rows so that each cell holds exactly one atomic value:

StudentNo	StudentName	CourseNo	CourseName
S1	Ahmed	CS101	Data Structures
S1	Ahmed	CS102	Algorithms
S2	Hussein	CS301	Databases

- **Atomicity Achieved:** Each cell now holds a single, indivisible value.
- **Primary Key:** The composite key $\{StudentNo, CourseNo\}$ uniquely identifies each row in the flattened relation.

> ⓘ **Note on Redundancy**
>
> While this table is now successfully in 1NF, the `StudentName` attribute is repeated for student `S1`. This structural redundancy is systematically addressed by moving to **Second Normal Form (2NF)**.

4.2 Second Normal Form (2NF)

A relation schema R is in Second Normal Form (2NF) if and only if:

- It is already in **1NF**.
- Every non-prime attribute is **fully functionally dependent** on every candidate key.

> ⓘ **Understanding Partial Dependencies**
>
> - **Scope:** 2NF is only relevant when a relation has a **composite primary key** (composed of two or more attributes). A relation with a single-attribute primary key is automatically in 2NF if it is already in 1NF.
> - **Partial Dependency:** This occurs when a non-prime (non-key) attribute is functionally dependent on only a *proper subset* of a composite candidate key, rather than the entire key.
>
> Second Normal Form eliminates these partial dependencies by breaking them out into separate relations.

📝 EXAMPLE 20: **Violating 2NF**

Consider the 1NF student enrollment table where the composite key is $StudentNo, CourseNo$:

StudentNo	StudentName	CourseNo	CourseName	InstructorNo	Grade
S1	Ahmed	CS101	Data Structures	I1	A
S1	Ahmed	CS102	Algorithms	I2	B+
S2	Hussein	CS301	Databases	I3	A-

Functional Dependencies Present:

- $StudentNo, CourseNo \rightarrow Grade$ (Full Dependency: both attributes are required to determine the grade).
- $StudentNo \rightarrow StudentName$ (**Partial Dependency**: `StudentName` depends only on `StudentNo`, not on `CourseNo`).
- $CourseNo \rightarrow CourseName, InstructorNo$ (**Partial Dependency**: course attributes depend only on `CourseNo`, not on `StudentNo`).

✅ Show Solution

To achieve 2NF, we decompose the table by extracting the attributes involved in partial dependencies alongside the respective subset of the key:

1. **Students Table** (Key: $StudentNo$)
   - Stores attributes dependent solely on the student identifier.

StudentNo	StudentName
S1	Ahmed
S2	Hussein

2. **Courses Table** (Key: $CourseNo$)

- Stores attributes dependent solely on the course identifier.

CourseNo	CourseName	InstructorNo
CS101	Data Structures	I1
CS102	Algorithms	I2
CS301	Databases	I3

3. **Enrollments Table** (Key: $StudentNo, CourseNo$)

- Stores the relationship and attributes fully dependent on the entire composite key.

StudentNo	CourseNo	Grade
S1	CS101	A
S1	CS102	B+
S2	CS301	A-

Result: All non-key attributes now depend on the *entire* primary key of their respective relations. Partial dependencies are eliminated.

4.3 Third Normal Form (3NF)

A relation schema R is in **Third Normal Form (3NF)** if and only if it is already in **2NF** and, for every non-trivial functional dependency $X \to A$, at least one of the following conditions holds:

- X is a **superkey** of R.
- A is a **prime attribute** (i.e., A is a member of some candidate key of R).

Core Rules & Key Concepts

- **Eliminates Transitive Dependencies:** A transitive dependency occurs when a non-key attribute determines another non-key attribute, forming a chain like $Key \to Y \to Z$ (where Y is not a superkey and Z is a non-prime attribute). 3NF systematically removes these structures to prevent update anomalies.
- **The Prime Attribute Exception:** Unlike Boyce-Codd Normal Form (BCNF), 3NF permits a non-key attribute to appear on the right-hand side of a dependency as long as that attribute belongs to a candidate key. This specific rule allows 3NF to preserve functional dependencies during decomposition where BCNF might otherwise fail.

EXAMPLE 21: **Violating 3NF**

Suppose we have a `Courses` relation schema that extends our 2NF design by also embedding instructor details:

$$Courses(CourseNo, CourseName, InstructorNo, InstructorName, InstructorLocation)$$

- **Candidate Key:** $\{CourseNo\}$
- **Functional Dependencies:**
  1. $CourseNo \to \{CourseName, InstructorNo\}$ (Key determinant)
  2. $InstructorNo \to \{InstructorName, InstructorLocation\}$ (Transitive dependency)

Why It Violates 3NF

- The functional dependency $InstructorNo \rightarrow \{InstructorName, InstructorLocation\}$ features a determinant ($InstructorNo$) that is **not a superkey** of the relation.
- Furthermore, the attributes on the right-hand side are **non-prime attributes** (they do not belong to any candidate key).
- This creates a transitive chain: $CourseNo \rightarrow InstructorNo \rightarrow \{InstructorName, InstructorLocation\}$.

✅ Show Solution

To bring the relation into 3NF, we decompose it by extracting the transitive dependency into its own relation:

1. **Courses Table** (Key: $CourseNo$)
   - Retains core course information and references the instructor via a foreign key.

CourseNo	CourseName	InstructorNo
CS101	Data Structures	I1
CS102	Algorithms	I2
CS301	Databases	I1

2. **Instructors Table** (Key: $InstructorNo$)

- Isolates attributes that depend directly on the instructor identifier.

InstructorNo	InstructorName	InstructorLocation
I1	Dr. Ali	Building A
I2	Dr. Sara	Building B

Result: Transitive dependencies are successfully eliminated. Updating an instructor's location now requires modifying a single row in the `Instructors` table, preventing data inconsistency.